Cluster Computing (2025) 28:669
https://doi.org/10.1007/s10586-025-05471-1

# A novel approach for enhancing reliability of microservices applications under dynamic workloads

Neha Kaushik[1] · Harish Kumar[1] · Vinay Raj[2]

Received: 9 November 2024 / Revised: 5 January 2025 / Accepted: 3 May 2025 / Published online: 3 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

## Abstract

The increasing need for independently developed and deployed web-based applications has made microservices architecture popular in the last decade. Its benefits have incentivized large IT companies like Netflix, Uber, and Twitter to adopt it. However, the distributed and loosely coupled nature of microservices based applications introduces reliability challenges under dynamic workloads. The failure of a microservice in the presence of dynamic workloads makes its dependent microservices unresponsive causing cascading failures and communication breakdowns resulting in reduced reliability. Effective workload handling and robust microservices communication mechanisms are essential to sustaining the reliability of microservices applications. This paper proposes a novel approach for the reliability improvement of microservice applications by focusing on these two issues. The proposed approach consists of four phases, the first phase assesses the reliability of microservices applications under varying workloads in the absence of the proposed approach and these results are used later for performing empirical analysis. The second and third phases of the proposed approach have focused on effective workload handling by predicting the microservices instances based on the forecasted workload. The fourth phase reduces the potential points of failure associated with microservices communication by placing interdependent microservices on the same node. To assess the performance of the proposed approach, an empirical investigation is performed on a case study application by employing synthetic and realistic workloads. The experimental results have shown that the proposed framework has significantly improved the reliability of the microservices application.

**Keywords** Microservices · Reliability · Machine learning · Workloads

## 1 Introduction

Distributed systems have gained a lot of attention in the past two decades as the demand for independent design and deployment of the web applications has increased. These systems play a key role in the timely delivery of diverse business needs by supporting continuous integration and continuous delivery of services. Microservices architecture is a distributed system evolved from monolithic and Service-Oriented Architectures (SOA) by enhancing their principles to deliver applications with enhanced modularity, scalability, and maintainability [1, 2]. Microservices architecture divides an application into small and independent services called microservices responsible for performing a single task [3, 30]. Each microservice can be designed using a different programming language or technology stack and can be deployed independently of other microservices residing in the system [4].

Harish Kumar and Vinay Raj have contributed equally to this work.

✉ Neha Kaushik
20001902008@jcboseust.ac.in

Harish Kumar
htanwar@jcboseust.ac.in

Vinay Raj
vinayraj@nitt.edu

[1] Department of Computer Engineering, J.C. Bose University of Science and Technology, YMCA, Faridabad, Haryana 121006, India

[2] Department of Computer Applications, National Institute of Technology Tiruchirappalli, Tiruchirappalli, Tamil Nadu 620015, India

Microservices use HTTP resource API for communication with other services present in the application [5].

Despite several benefits, performance, reliability, and security are the challenges that need to be addressed in order to improve the quality of Microservices Architecture (MSA) [6–8]. Distributed systems like microservices face silent communication breakdowns which makes the whole application unavailable [9]. Most of the existing studies have improved the reliability of microservices applications either by focusing on varying workloads [10] or on the interdependencies between the microservices [11]. In our work, both varying workloads along with the interdependencies between the microservices are considered. According to the IEEE standard, Reliability is characterized as the capacity of an application to execute its designated functions within specified conditions throughout a defined duration [11]. Dynamic workloads can lead to resource exhaustion and make microservices applications unresponsive [12]. The services in an MSA based application are loosely coupled i.e. they rely on other services for some functionalities. So, failure of a microservice can have a cascading effect on its dependent services and can lead to a breakdown in communication between services and affect the overall reliability of the application [9, 13, 14]. Microservices architectures need to be resilient and capable of recovering from such failures because if failures occur and the system is unable to recover quickly, it can lead to prolonged periods of reduced reliability. Hence, a reliability improvement strategy should be capable of handling dynamic workload situations and also should be capable of handling the failures that can occur because of communication breakdown.

This paper addresses the challenge of enhancing reliability by proposing a novel approach for improving the reliability of microservices applications under dynamic workloads. If an application receives sudden spikes in resource demands and the fluctuations in workload patterns refer to dynamic workload. The proposed approach helps in improving the reliability of microservices applications by efficiently handling the dynamic workload and reducing the chances of failures caused by microservices communication breakdown. The following research questions are framed to articulate the motivation behind this study and to offer insights into the proposed framework: RQ 1: What are the factors that affect the reliability of the microservices application? RQ 2: How can the reliability of MSA based applications be improved? RQ 3: What is the role of machine learning in improving the reliability of microservices applications?

To address RQ1, a detailed literature survey is conducted to identify the factors impacting the reliability of microservice applications. A four phase approach is proposed to address the RQ2 and different ML algorithms are used for effective handling of dynamic workloads as part of RQ3. The proposed approach consists of four phases and in each phase an algorithm is stated for achieving the goal as shown in Fig. 1.

In the proposed approach, phase 1 is the reliability assessment of the microservices application. In this phase, an algorithm for assessing the default reliability of the application against the dynamic workload has been developed. In phase 2, a machine learning model is trained to predict the number of instances required for handling the user requests successfully. Phase 3 of the proposed framework focuses on the prediction of the workloads for the next time interval and also predicts the number of microservice instances required for handling that workload. This way the application becomes reliable by taking proactive action for handling the dynamic environment and the user requests are serviced without failure. Proactive action makes the application capable of handling the upcoming workload without any failure. The last phase of the proposed approach reduces the potential points of failure associated with microservices communication by placing the predicted instances on the nodes such that the microservices that communicate more frequently are placed on the same node. This strategic placement of microservices reduces network latency and the risk of communication breakdowns, which are essential during dynamic workloads. Relying less on network communication makes the system more robust to failures and reduces the likelihood that the failure of one service will affect other services.

To summarise, this paper makes the following contributions to achieve the proposed objective:

- A literature survey is conducted to identify the factors affecting the reliability of microservices applications.
- Proposed a novel approach for improving the reliability of microservices application under dynamic workload.
- An efficient machine learning model is used to predict the microservices instances for upcoming workloads.

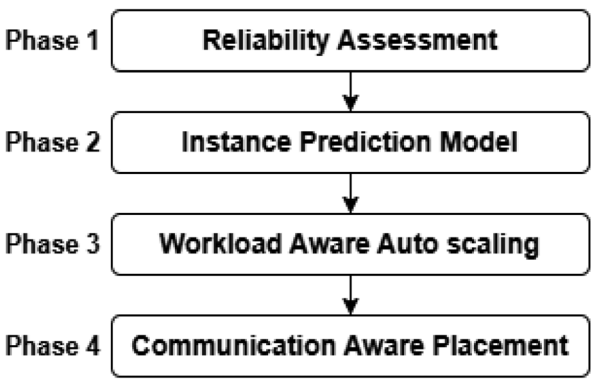



**Fig. 1** Phases in the proposed approach

- Extensive comparison of the reliability of the case study application under dynamic workloads with and without using the proposed approach.

The rest of the paper is structured as follows: Sect. 2 presents notable works on the reliability of service oriented systems and microservices. The proposed approach is discussed in Sect. 3. The machine learning model for microservices instance count and workload prediction is discussed in Sect. 4. Empirical Analysis of the proposed framework is discussed in detail in Sect. 5. The threats to validity of the proposed work are presented in Sect. 6 and Sect. 7 concludes the paper.

## 2 Literature review

Microservices, also referred to as services, are the fundamental architectural elements of Microservices Architecture. In this section, firstly the factors affecting the reliability of microservices applications are identified and then a detailed study is conducted on the existing work related to the reliability of microservices applications. The detailed study helped to identify the existing gaps for reliability improvement and then further research was conducted to determine strategies for addressing these gaps effectively.

Wang et al. [6] have found out that performance, reliability, and security are the quality attributes that require more attention in order to improve the overall quality of microservices applications. Jagadeesan and Mendiratta [9] discussed how loose coupling among microservices affects the availability of the overall application. The authors have proposed a service mesh and side cars based framework to analyse the reliability of microservices applications at scale. Shafi et al. [10] have improved the reliability and performance of microservices applications by performing dynamic scaling of resources for handling the workload. The author has shown how the improper handling of workload can lead to reduced reliability of microservices applications. Li et al. [13] demonstrated how complex dependencies among tasks can make an entire microservices application unavailable, thereby degrading the user experience. Consequently, the authors suggest focusing on the reliability parameter during the execution of microservices applications. Ding et al. [15] have emphasized considering the runtime environment and software dynamic behaviour for reliability prediction and reliability improvement of Service oriented Systems. The author has proposed a framework that uses run-time data such as response time to predict and improve the reliability of Service Oriented Systems. Francesco et al. [7] have identified that achieving reliability in MSA based applications is difficult because of their distributed architecture. Malki et al. [16] have considered the workload configurations and predefined rate limits for predicting their effect on the reliability of the microservices, The author found that limiting the API rate significantly improves the reliability of microservices applications but also impacts the performance and scalability of the application. The insights gained from these studies underscore the significance of the reliability quality attribute in enhancing the overall quality of microservices applications. They reveal that to improve reliability, it is essential to efficiently manage workload and address communication failures among microservices, which are inherent to the distributed architecture of these applications. After identifying the factors, the subsequent study focused on existing research addressing the reliability of microservices applications and service oriented systems.

Pietrantuono et al. [17] have proposed EMART, an operational reliability assessment method for microservices applications. The author suggested using the resulting data for reliability improvement of the applications. Liu et al. [18] improved the reliability of microservices applications by placing dependent microservices together. In future work, the author suggested including environmental diversity to predict the reliability of the application. Song and Tilevich [19] have proposed a data flow based programming model for better resource utilization and improved reliability. The author has focused on the development phase reliability improvement while ignoring the operational reliability of the application. Camilli et al. [20] have proposed a novel approach, MIPaRT for testing the performance and reliability of microservices applications under varying workloads. Shafi et al. [10] have proposed CDAScaler, a novel approach to allocate CPU resources to each container of microservices applications during the scaling operations. The proposed approach leads to improved reliability and performance of microservices applications. Wang et al. [21] have used Convolutional Neural Networks (CNN) and Long Short Term Memory (LSTM) for predicting the reliability of service oriented systems. The authosr have used response time and throughput of component services at different time intervals for making the predictions. After reviewing the existing research on the reliability of microservices applications, it is evident that there is a significant gap in work aimed at improving reliability. Most existing studies have improved the reliability of microservices applications by either focusing on varying workloads or on the interdependencies between microservices necessary for task completion. To the best of our knowledge, no current research addresses both efficient workload management and communication failures among microservices. This gap has driven us to develop strategies focusing on

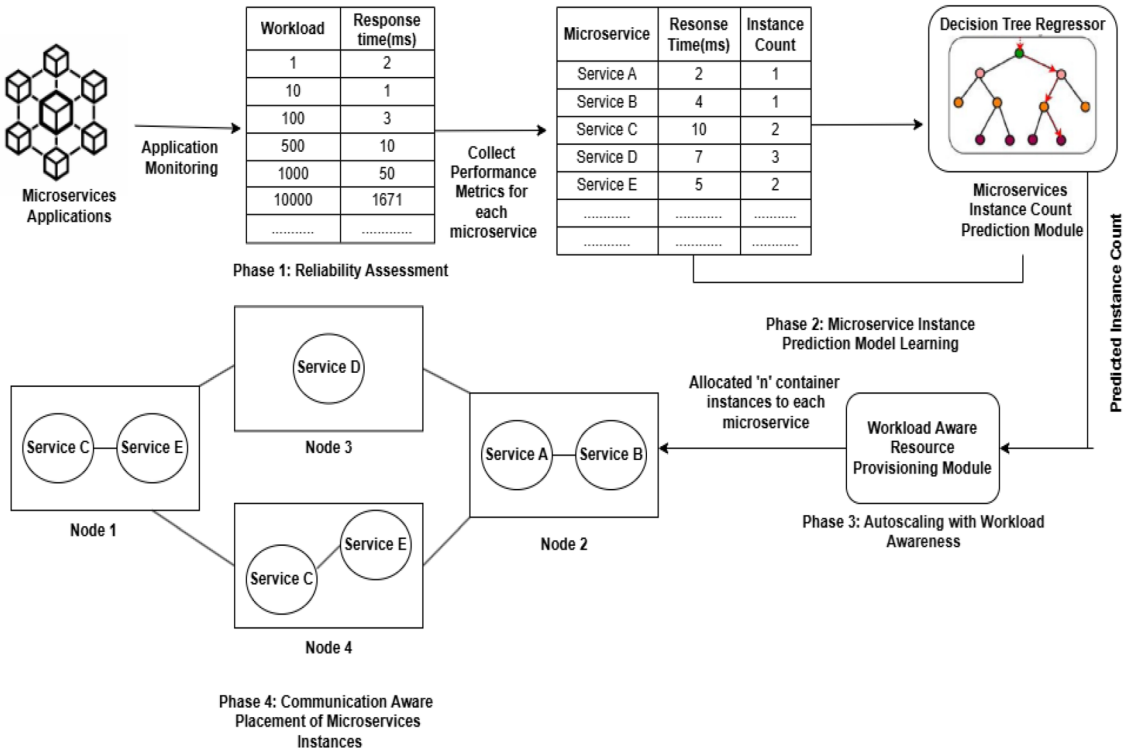



**Fig. 2** Proposed Framework

effective workload handling and robust microservices communication mechanisms for sustaining the reliability of microservices applications. Now, further studies will focus on the approaches used in existing works for managing varying workloads and addressing failures arising from the communication between microservices required to complete a task.

Abdullah et al. [12] have proposed an intelligent auto scaling method that can predict the burst in workload and can make autoscaling decisions by provisioning the resources to the microservices. The problem with this work is that the author used a microservice application consisting of a single microservice. Yan et al. [22] have used Bi-LSTM for predicting the workload of a microservices application. The results show the proposed load prediction method results in the effective utilization of resources while meeting all service level agreements. Feng et al. [23] have proposed FAST, a novel prediction model for predicting the dynamic workloads of microservices applications. The proposed model considers the relationship between different workload changes and proper window size, including trend correlation, time correlation, and random fluctuations. In the future, the prediction model can be used to improve the quality of microservices applications. Xu et al. [24] have proposed a CoScal approach to improve the performance of microservices applications while efficiently utilizing the resources and keeping the cost minimum. The proposed approach used a neural network based workload prediction approach and performed the scaling of resources. In the paper [25], the authors Luo et al. discuss that the colocation of dependent microservices reduces the response time and network communication overhead of microservices application. Joseph and Chandrasekaran [26] performed the interaction aware placement of microservices for reducing communication breakdowns among microservices and improving the response time of the application. It is evident from the study that predicting workload and taking appropriate actions based on these predictions are crucial for effective workload management. The study also suggests that colocating dependent microservices can significantly improve application response time by reducing potential communication breakdowns. The proposed approach improves the reliability of microservices application by focusing on workload management and colocating dependent microservices.

## 3 Proposed approach

A machine learning based framework is proposed for improving the reliability of the microservices architecture based application. The proposed framework mainly consists of four phases. Figure 2 shows the steps used by the

proposed framework for improving the reliability of the microservices application.

The first phase deals with the reliability assessment of microservices application. The resource prediction model has been trained in the second phase. The third phase of the framework performs bursty workload aware horizontal scaling of the resources by using the instance prediction model trained in phase 2 and also by predicting the workload. In the fourth phase, the instances predicted in the third phase are placed on the nodes based on their communication with other microservices and their resource requirement. All four phases along with the four algorithms are discussed in detail in the following subsections.

### 3.1 Reliability assessment

In the first step, the reliability of microservices application is estimated. The response time of a user request is used as a basic block for determining the reliability. If the response time of a user request is greater than the threshold value then that request is considered a failed request. The Algorithm 1 focuses on the reliability assessment of the microservices application in the presence of dynamic workloads consisting of bursty and non bursty workloads. The list of notations used in Algorithm 1 are represented in Table 1.

Algorithm 1 aims to determine the reliability ($x$) of microservices application, Response Time of the application (res) and Number of failed requests ($n_f$). For this purpose, Application monitoring interval duration ($T_m$), Number of Requests sent ($n$), and Application Threshold Response time ($R_t$) are the required variables. Initially, the algorithm focuses on the deployment of the case study microservices application. In the beginning, no workload is applied on the application, hence the number of requests sent ($n$) and number of failed requests ($n_f$) is 0 in line 3. Subsequently, a workload is applied to the application for a duration of $T_m$, during which the value of $n$ is incremented by 1 for each user request. Simultaneously, the response time (res) is measured and recorded for each corresponding $n$. Line 8 to line 12 focuses on calculating the reliability of the application. For this purpose, the response time (res) of each user request is compared with the Application Threshold Response time ($R_t$), if the response time of a user request (res) is greater than the Threshold Response time ($R_t$) then that user request is considered as failed request and for each failed request the value of nf is increased by 1. After calculating failed requests, the reliability ($x$) of the microservices application is calculated using the formula mentioned in line 13.

**Table 1** List of notations used in Algorithm 1

| Notations | Definition |
|---|---|
| $T_m$ | Application monitoring interval duration |
| $n$ | Number of Requests sent |
| $R_t$ | Application Threshold Response time |
| $n_f$ | Number of failed requests |
| $res$ | Response Time of the application |
| $x$ | Reliability |

**Algorithm 1** Reliability assessment of the Microservices application

**Input:** $T_m$, $n$, $R_t$
**Output:** $n_f$, $res$, $x$
1: **Begin**
2: deploy the microservices application
3: initialize n = 0, $n_f$=0
4: **while** $T_m$ **do**
5: $n = n + 1$
6: calculate res of each n
7: **end while**
8: **for** $i \leftarrow 1, n$ **do**
9: **if** $res > R_t$ **then**
10: $n_f \leftarrow n_f + 1$
11: **end if**
12: **end for**
13: $x = 1 - \frac{nf}{n}$
14: **return** $x$
15: **End**

**Table 2** List of notations used in Algorithm 2

| Notations | Definition |
|---|---|
| $N$ | Number of microservices in the application |
| $T_m$ | Application monitoring interval duration |
| $MC_i$ | Number of allocated container instances of each microservice |
| $r_{threshold}$ | Threshold Response time of each microservice |
| $MC'_i$ | Updated container instances for each microservice |

The value of x will be between 0 and 1 and the higher value of x will represent the higher reliability of the application. The results of the assessment will later be used for comparing the performance of the proposed framework of reliability improvement. The Algorithm 1 is applied only for accessing the reliability of microservices application with and without the proposed approach.

The output of Phase 1 serves as the foundation for a comparative analysis of the microservices application with and without the proposed approach. Initially, the case study application was assessed for reliability without the proposed solution, followed by a reevaluation after integrating all phases. This approach not only demonstrated how reliability was evaluated but also eliminated the need for redeployment to obtain baseline results without the proposed method. Although the output of Phase 1 is not directly used in subsequent phases, it remains integral as a preparatory step for establishing a meaningful comparison.

## 3.2 Microservice instance prediction model learning

This phase is responsible for the designing of a machine learning model that can predict the number of container instances of each microservice needed to handle the user request such that the response time remains less than the Threshold Response time of each microservice (rt). A reactive approach is used to collect the performance traces that will be used later to train the response time model. Performance traces are collected using a distributed tool, Zipkin supporting OpenTracing. The first step for collecting the traces is to apply workload on the application using locust.io tool. The workload progressively increases by generating concurrent user requests, leading to a significant saturation of the microservice's response time. Now a reactive autoscaling method is used by Algorithm 2 to scale in and scale out the container instances of all microservices based on the response time. This algorithm scales out the allocated container instances by adding one container instance whenever the 95th percentile of response time is greater than the user defined Threshold Response time (rt) and scale-in the allocated resources whenever three consecutive intervals show the 95th percentile of the response time less than rt/2. Algorithm 2 is similar to [27] by extending it for scale-out and scale-in on the basis of response time. The list of notations used in Algorithm 2 is represented in Table 2.

**Algorithm 2** Reactive Autoscaling Method

**Input:** $N$, $T_m$, $MC_i$, $r_{threshold}$
**Output:** Updated container instances $MC'_i$ and Performance metrics (CPU, mem, response time)

**Begin**
**while** $True$ **do**
  wait for $T_m$ seconds and monitor the call graph and performance of application
  **for** each microservice $i \leftarrow 1, N$ **do**
    Extract $r_t$, $95^{th}$ percentile of response time at current time interval t.
    **if** $r_t > r_{threshold}$ **then**
      $MC'_i$ ++
    **else if** $r_t \& r_{t-1} \& r_{t-2} < (r_{threshold}/2)$ **then**
      $MC'_i$ −−
    **end if**
  **end for**
t = t + 1
**end whileEnd**

**Table 3** List of notations used in Algorithm 3

| Notations | Definition |
|---|---|
| $N$ | Number of microservices in the application |
| $T_m$ | Application monitoring interval duration |
| k | Workload window size |
| I | Microservice Instance Prediction Model |
| W | Workload Prediction Model |
| B | Burst detection function |
| $r_{threshold}$ | Threshold Response time of each microservice |
| $MC_i$ | Number of allocated container instances of each microservice |
| $MC_i^{'}$ | Updated container instances for each microservice |

In Algorithm 2, Line 3 to Line 12 are executed as long as the workload is applied on the application. After waiting for some $T_m$ time the performance metrics such as response time, CPU and memory utilization along with microservices call graphs are collected in Line 3. Now for each microservice present in the application extract the 95th percentile of response time ($r_t$) of all user requests that came in monitoring time. Now compare the $r_t$ of each microservice with their threshold Response time ($r_{threshold}$). Now if $r_t$ is greater than $r_{threshold}$ there is a need to scale the microservices instance by 1 else if $r_t$ of the previous two intervals and current interval is less than the half of current $r_{threshold}$ then the number of instances is decreased by 1 to save resource wastage. It can be said that Lines 5 to 9 focuses on collecting the number of instances required to handle different workloads such that the response time of each microservice remains less than the threshold response time value in a resource efficient manner. Line 11 represents the change in time interval i.e. next time interval and then again Line 3 to Line 12 are repeated.

The collected data is used to train the microservice instance prediction model. From various supervised machine learning algorithms, the Decision Tree Regression model has been used for predicting the number of microservices instances needed for handling varying workloads. From the total dataset, 80% of the collected dataset has been used for training the model and 20% of the dataset has been used for testing the model.

$$I : (w, r_{threshold}) \Rightarrow MC_i^{'} \quad (1)$$

### 3.3 Autoscaling with workload awareness

This phase of the proposed framework concerns the autoscaling in the presence of the bursty workload. To decide the level of intensity of burst and to allocate the number of containers for each microservice based on it, an algorithm is presented as shown in Algorithm 3. The list of notations used in Algorithm 3 is represented in Table 3.

**Algorithm 3** Autoscaling with Bursty Workload Awareness

**Input:** $N$, $T_m$, $k$, $I$, $W$, $B$, $MC_i$, $r_{threshold}$
**Output:** Newly Provisioned container instances of each microservice $MC_i'$

```
1: Begin
2: isBurst = F                          ▷ representing the presence of bursty workload
3: while True do
4:     for each microservice M_i, i ← 1, n do
5:         Wait for T_m seconds
6:         w_{t+1} = W(w_t, w_{t-1}, ··· , w_{t-k-1})
7:         ñ_{t+1} = I(w_{t+1}, r_threshold)   ▷ Predict the number of microservice instances
8:         y = B(w_t, w_{t-1}, ··· w_{t-k-1})
9:         n_max = MAX(ñ, ñ_{t-1}, ··· , ñ_{t-k-1})
10:        if y == 1 and isBurst == F then
11:            isBurst = T
12:            n_c = n_max
13:        else if y == 1 and isBurst == T then
14:            n_c = n_max
15:        else if y == 0 and isBurst == F then
16:            n_c = ñ_{t+1}
17:        else if y == 0 and isBurst == T then
18:            if ñ_{t+1} < n_max then
19:                isBurst = F
20:                n_c = ñ_{t+1}
21:            else
22:                n_c = n_max
23:            end if
24:        end if
25:        if n_c > MC_i then
26:            MC++
27:        else
28:            MC−−
29:        end if
30:    end for
31:    t = t + 1
32: end while
33: End
```

Algorithm 3 represents the autoscaling of resources based on a bursty workload. Because initially there is no bursty workload, the value of variable *isBurst* is equal to *F* in line 2. Forecasting the workload and determining the optimal number of instances required to handle it efficiently, from line 3 to line 32, ensures seamless performance while the application receives user requests continuously. Now for each microservice $M_i$ where $i$ varies from 1 to N denoting the number of microservices, wait for $T_m$ seconds. The $T_m$ seconds represents the time taken to predict the workload and the instances. In Line 6 the workload is predicted using a machine learning algorithm as discussed in Sect. 4. The number of microservices instances is predicted in line 7 using Algorithm 2 discussed in Phase 2 of the proposed approach. In line 8, the Burst is detected using function B() which uses Sample Entropy as discussed in [28]. If there is a burst then the value of $y$ will be 1 else it will be 0. The prediction is made by observing the workload of $k$ windows. The condition in line 10 represents the bursty workload predicted for the next time interval and no burst at the current moment. So in that case the number of instances $n_c$ will be $n_{max}$ i.e. the maximum instance count predicted in the last $k$ windows and *isBurst* is set to *T* from *F*. The condition in line 13 addresses a scenario where a bursty workload is forecasted for the upcoming time interval, and the system is already managing a bursty workload at the current moment. In this case, only the value of $n_c$ is changed to maximum instance

count $n_{max}$ and the value of *isBurst* remains the same. Line 15 represents the case when no burst is predicted for the next time interval and at the current time there is no burst. In such a case there is no need to allocate maximum instance count, the instance count $n_c$ will be equal to the number of instances predicted using machine learning model $I$ i.e. $\tilde{n}_{t+1}$ and the value of *isBurst* remains unchanged. Line 17 represents the condition where the bursty workload is present in the current interval but in the future it will not be there. So in that case the value of $n_c$ will depend on whether the predicted instances $\tilde{n}_{t+1}$ is less than $n_{max}$. The variable $\tilde{n}_{t+1}$ shows that in the future there will be a lesser workload to handle, so for efficient resource utilization, it is better to use the $\tilde{n}_{t+1}$ value and assign $F$ value to *isBurst*. In other cases, the value of $n_c$ will be $n_{max}$, and the value of *isBurst* remains unchanged because we still have a bursty workload. Line 25 checks whether the predicted number of instances is greater than the allocated number of instances of microservices, if true then more instances are allocated accordingly otherwise the allocated instances are reduced accordingly.

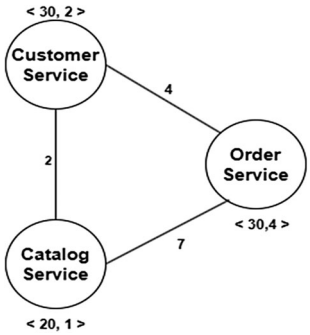



**Fig. 3** Interaction Graph of Case Study Application

## 3.4 Communication aware placement of microservice instances

Network latency affects the response time of the application. If the network latency can be reduced then the overall response time of the microservice application can also be reduced significantly. Algorithm 4 achieves this purpose by using an interaction graph as input. This interaction graph is a graph with a single instance of each microservice deployed on different nodes. Vertex V of the interaction graph represents the microservice and its CPU and memory need and edge E represents the communication link between the microservices. An edge weight is assigned to each edge. The edge weight is calculated by observing the number of times two microservices communicate in an interval of time. The result obtained after observation is then divided by 2 because communication was bidirectional and is assigned as edge weight on that particular edge.

Figure 3 depicts the interaction graph of the case study application. The case study application consists of three microservices Customer service, Catalog service and Order Service. Each node in Fig. 3 represents a microservice of the case study application. The nodes in Fig. 3 are assigned a weight <mem_req, cpu_req> where mem_req represents the amount of memory required by that microservice for successful execution of the task and cpu_req is representing the amount of CPU required by that microservice for successful execution of the task. Here the <mem_req, cpu_req> weights are assigned to the nodes using Algorithm 2 where the application is analysed for a specific period of time to collect the data. The arcs in Fig. 3 represents the bidirectional communication link and an edge weight is assigned using Algorithm 2 based on the communication between the respective nodes. Table 4 contains the list of notations used in Algorithm 4.

**Table 4** List of notations used in Algorithm 4

| Notations | Definition |
|---|---|
| $G=<V,E>$ | Interaction Graph with vertices V and Edges E. V is the set containing all vertices of graph G and E is the set containing all edges of graph G |
| $V^{cpu_need}$ | It represents the CPU need of each vertex V |
| $V^{mem_need}$ | It represents the memory need of each vertex V |
| W | Edge Weight |
| S | Set of nodes/devices |
| count | Array containing predicted instances of all microservices |

**Algorithm 4** Communication aware placement of microservices instances

**Input:** Interaction Graph $G$ with weight $V$, edge weight $W$, set of nodes/devices $S$, count array
**Output:** Mapping of V to S
1: **Begin**
2: Initialize $S^{cpu_res\&mem_res} > 0$
3: Create an array ASSIGNED and initialize it with value zero.
4: sort W and S in decreasing order
5: **while** ASSIGNED[]==0 **do**
6: $x_{ij}$ = Extract MAX(W)
7: D = Extract MAX(S)
8: **while** $x_{ij}$ **do**
9: **if** $V_i^{cpu_need} + V_j^{cpu_need} < D^{cpu_res} and V_i^{mem_need} + V_j^{mem_need} < D^{mem_res}$ **then**
10: place $V_i$ and $V_j$ on D
11: update array ASSIGNED[] for i and j
12: update count[i] and count[j]
13: update $D^{cpu_res\&mem_res}$ by subtracting *cpu_need* and *mem_need* of $V_{ij}$
14: **end if**
15: **end while**
16: **if** $V_i^{cpu_need} < D^{cpu_res} and V_i^{mem_need} < D^{mem_res}$ **then**
17: place $V_i$ on D
18: update array ASSIGNED[] for i
19: update count[i]
20: update $D^{cpu_res\&mem_res}$ by subtracting *cpu_need* and *mem_need* of $V_i$
21: **end if**
22: **end while**
23: **End**

The residual amount of CPU and memory available on the nodes should be greater than 0 and are presented by S. The same thing is written in line 2 of Algorithm 4. In line 3, an array named ASSIGNED is created to keep track of whether the number of instances of each microservices are adjusted according to the prediction made by using Algorithm 3 or not. Initially, no microservice instances are adjusted so it is 0. Now the Edge weights (W) and set of nodes (S) are sorted in decreasing order in line 4. Now line 5 is responsible for the communication aware placement of microservices and hence it continues its execution until all the predicted instances of all microservices are not placed i.e. until ASSIGNED[] does not become 0. Line 6 finds the microservices with the highest edge weight means the microservices communicate more often. Line 7 finds the device with the highest available CPU and memory resources. Line 8 focuses on optimizing resource allocation by attempting to colocate the microservices whose CPU and memory needs can be fulfilled by the residual resources available on the nodes/devices. Line 9 checks if the CPU and memory needs of both the microservices can be fulfilled by the node with the highest available resource, if yes then they are placed on device D and the array ASSIGNED and count is updated accordingly. And also the residual resources on device D are updated. Line 14 is responsible for placing all the microservices instances that failed for communication aware placement by selecting random devices that can fulfill their needs.

Figure 4 represents the communication aware placement of microservices. In this graph, the nodes represent the server or device on which the services are deployed. Based on the residual CPU and Memory space available on each node the microservice instances are deployed on them. The placement of microservices based on their interaction reduces network delays. The size of the graph can increase with the increase in the number of microservice instances. The order service and catalog service have the highest edge weight so they are placed on the same node. The next largest edge weight links order service and customer service so they are kept on the same node. Customer service and catalog service communicate with each other through API.

## 4 Instance and workload prediction model

In this paper, various machine learning techniques like linear regression, support vector regression and decision tree regression have been used for predicting the workload and microservices instance count. The final selection of the machine learning technique has been made based on the magnitude of relative error (MRE). The machine learning technique leading to minimum MRE is selected as the final technique for workload prediction and microservices instances prediction. MRE is calculated as given below.

$$MRE = \frac{ActualResponseTime - EstimatedResponseTime}{ActualResponseTime} \quad (2)$$

### 4.1 Microservice instance prediction model

Initially, the data set for training the machine learning model is collected by using the approach discussed in Phase 2 of Sect. 3. Out of the total data set, 80% of the collected dataset has been used for training the model and 20% of the dataset has been used for testing the model. For learning an appropriate instance prediction model, Linear regression (LR), Support Vector regression (SVR) and Decision Tree Regression (DTR) machine learning methods have been evaluated. The results show that decision tree regression performed better in comparison to the other two methods. Table 5 represents the evaluation results.

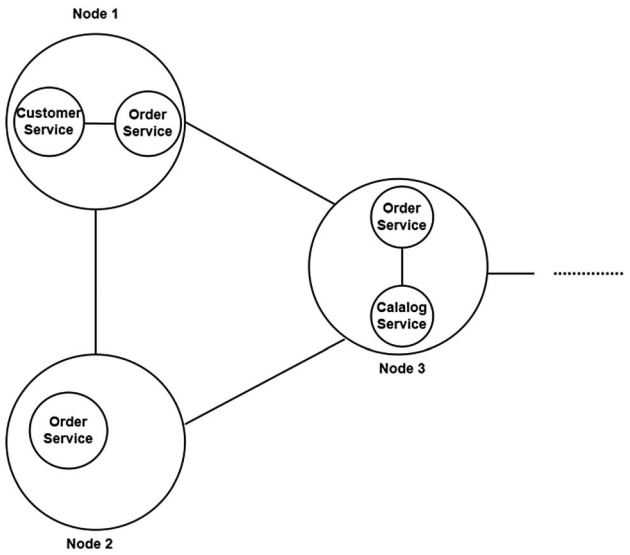



**Fig. 4** Communication aware placement of services

**Table 5** MRE for microservice instance prediction model

| Method | LR | SVR | DTR |
|---|---|---|---|
| MRE | 0.208 | −0.278 | 0.154 |

### 4.2 Workload prediction model

The task of the workload prediction model is to predict the workload for the next time interval t+1 by observing the workload of the past few $k$ time intervals. For different values of k, different machine learning models like Linear regression (LR), Support Vector regression (SVR) and Decision Tree Regression (DTR) techniques have been used for predicting the workload. The window size k and machine learning model with minimum MRE are selected for workload prediction. Wikipedia workload has been used for training and testing of machine learning models. The results indicate that linear regression outperformed the other two methods. As shown in Table 6, linear regression has the lowest mean relative error (MRE) for k=12.

## 5 Empirical analysis

In this section, the performance of the benchmark application integrated with the proposed framework is evaluated. The case study application *Microservice Consul* is deployed, managed and scaled using docker, docker compose and docker swarm. Locust.io, an open-source load testing tool has been used for performance testing of the application and Zipkin, an open-source distributed tracing system used to trace calls across microservices. With the help of Zipkin, response time of each microservice for a user request can be collected. Prometheus, an open-source monitoring system is used for collecting the CPU and memory utilization metrics of each microservice instance. Grafana, an open source analytics & monitoring solution was used on top of Prometheus to offer visualization functionalities for data. For training the workload prediction model and for evaluating the performance of proposed framework a real time workload [29] is used.

### 5.1 Case study application

A microservices application named Microservice Consul, where users can buy books, is used to evaluate and compare the performance of the proposed framework. Microservice Consul consists of three microservices order service, customer service and catalog service. The Customer Service handles customer related data, the Catalog Service manages the items in the catalog, and the Order Service is responsible for orders processing by using Catalog service and Customer service.

**Table 6** MRE for workload prediction model

| Window Size / Method | LR | SVR | DTR |
|---|---|---|---|
| k=4 | 0.199 | 0.210 | −0.215 |
| k=8 | 0.185 | 0.225 | −0.237 |
| k=12 | 0.173 | 0.240 | −0.250 |

The Case study application consists of Customer service, Catalog service, and Order service as shown in Fig. 5. The user accesses the application on its computer screen using a web browser. The user sends a request to the application by clicking an option on the website. The request is forwarded to the application gateway. The application gateway routes user requests to the appropriate service. Each service within the case study application is responsible for executing a specific task. The application gateway selects a microservice for processing the user request based on the kind of request the user has sent and the tasks for which a microservice is responsible. For example, if a user wants to make a profile on the application or wants to update their profile then in that case those user requests will be forwarded to customer service, because customer service is responsible for handling user data. Once the user request is routed to a microservice, that microservice is tasked with processing it. Now if a microservice wants to communicate with other microservices of the application then it can be done with the help of the application gateway. After processing the user request, an HTTP response is sent to the application gateway. After that, the response is forwarded to the web browser. The Fig. 6 represents the interactions and inter relationships among customer, catalog and order service.

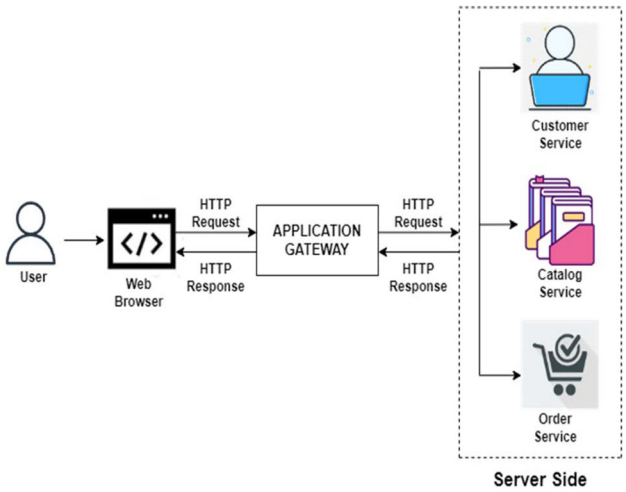



**Fig. 5** Microservice Architecture Based Case Study Application

To assess the proposed approach better, the case study application is initially deployed using Docker Swarm and we refer to this application as *Without the proposed approach* during result analysis. Another version of the same case study application is deployed by integrating the proposed algorithms with monitoring and logging processes. This version of the application is referred to as *With the proposed approach* during result analysis.

## 5.2 Experimental parameters

In the experimental evaluation, to find the most appropriate value of monitoring interval duration ($T_m$) and window size (k) the proposed framework was implemented for different values. The results lead to minimum response time and minimum MRE for $T_m$ = 90 s, duration of each interval= 8 min and k = 12. The user defined attribute $r_{threshold}$ is assigned the value 150 milliseconds during implementation. In the experimentation, several different values were assigned and tested but the values defined in this section were found to perform better in terms of response time and resource utilization.

## 5.3 Results and discussion

This section discusses the evaluation results presented in tables for each microservice and the reliability outcomes of the case study application with and without the proposed framework. Since the reliability of the applications depends on the number of failed requests, during the analysis, we presented the details of processed requests and failed requests along with the cost of resources for each service of the case study application. However, in the end, the reliability of the application is compared without and with the proposed approach.

The results of customer service are presented in Table 7, which consists of four columns. The first column lists various synthetic workloads (SWL) generated using the locust.io tool and a real-time Wikipedia workload available online. Any number of SWL can be generated using locust.io but in this experimentation, only six SWLs named SWL 1, SWL 2, SWL 3, SWL 4, SWL 5 and SWL 6 are generated. Each SWL varies from the others because different numbers of requests can be sent to the application for different durations of time. For the analysis, SWL1 is considered with 500 requests, SWL2 with 1000 requests, SWL3 with 3000 requests, SWL4 with 5000 requests, SWL5 for 7000 and SWL6 with 10000 requests per second. The second column of Table 7 shows the number of requests processed under these workloads, the third column indicates the percentage of user requests that failed to respond on time, and the final column provides a cost analysis, evaluating resource usage by the case study

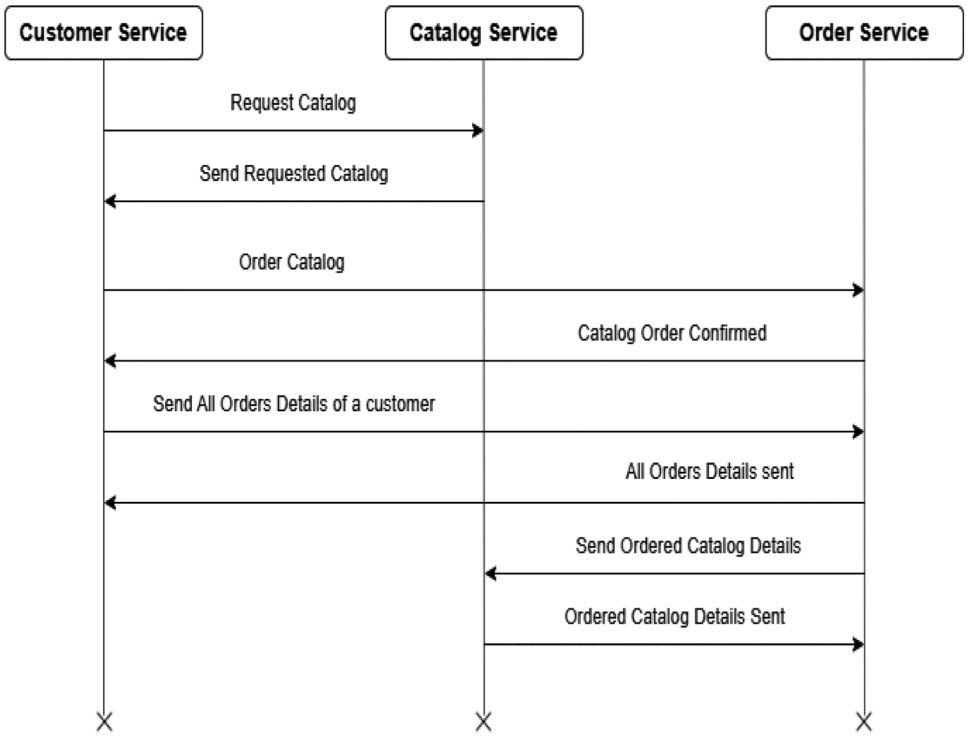



**Fig. 6** Illustration of the interactions and interrelatipnships among microservices

application in presence and absence of the proposed framework. The results show that the application successfully responds to more user requests when using the proposed framework.

Graphical comparison between the number of successfully executed user requests of customer service using the proposed framework and without using the proposed framework is represented in Fig. 7. It is clear from the graphical representation that the proposed framework is executing more number of user requests successfully. In the case of SWL 2, SWL 3 and SWL 5 the margin between the number of successfully executed user requests for customer service using both approaches is the least.

The graphical comparison of the number of failed requests of customer service in the presence and absence of the proposed framework is represented in Fig. 8 and it is observed that the proposed framework is resulting in a reduced number of failed requests.

The cost comparison of customer service with and without the proposed approach is represented in Fig. 9. The graphical depiction of costs indicates that the proposed framework increases costs in all scenarios except for SWL 4. When SWL 4 is applied to the application then the proposed framework leads to reduced cost but in all other workload cases, the proposed framework results in increased cost.

**Table 7** Simulation Results of Customer Service

| Workload | Processed Requests (%) | | Failed Requests (%) | | Cost (%) | |
|---|---|---|---|---|---|---|
| | Without Using Model | With Model | Without Using Model | With Model | Without Using Model | With Model |
| SWL 1 | 97.01 | 98.05 | 2.99 | 1.95 | 1.3 | 1.35 |
| SWL 2 | 98.8 | 99.01 | 1.2 | 0.99 | 2.56 | 3.5 |
| SWL 3 | 96.54 | 96.78 | 3.46 | 3.22 | 3.06 | 5.73 |
| SWL 4 | 91.6 | 94 | 8.4 | 6 | 5.98 | 5.01 |
| SWL 5 | 92.39 | 92.68 | 7.61 | 7.32 | 8.56 | 9.58 |
| SWL 6 | 93.91 | 95.57 | 6.09 | 4.43 | 9.05 | 10.11 |
| Wikipedia | 90.17 | 93.95 | 9.83 | 6.05 | 10.52 | 13.72 |

The simulation results of the Catalog service are presented in Table 8. The results show that the number of failed requests using the proposed framework has been reduced. In all scenarios, whether it is a synthetic workload or real time workload situation, the proposed framework increases the cost while improving the processed request count.

The comparison of a number of successfully executed user requests by catalog service in the presence and absence of the proposed framework is demonstrated in Fig. 10. It is clear from Fig. 8 that the absence of the proposed framework results in a reduced number of successfully executed services.

The graphical comparison of a number of failed user requests of catalog using and without using a proposed framework is depicted in Fig. 11. It is clearly visible from the figure that the absence of the proposed framework is reducing the number of processed requests. In the case of SWL 2 and SWL 4 the number of failed requests is almost the same in both cases.

Graphical based comparison of cost of catalog service in the presence and absence of the proposed framework of reliability improvement is shown in Fig. 12. In the case of SWL1, SWL 3, SWL 4, and SWL 6 the cost of using and not using proposed framework are almost the same but in the remaining cases of workload, the cost without using proposed framework is less than the cost of resulting from using the proposed framework.

The number of processed requests, failed requests and cost comparison of Order service are represented in Table 9. The results show that in the case of real time workload, the number of processed requests is almost the same for both models while the cost is higher in the case of the proposed framework.

Graphical based representation of processed requests of order service with and without proposed model is shown in Fig. 13. In the presence of various workloads, the number of processed requests using the proposed approach is higher than the number of user requests processed

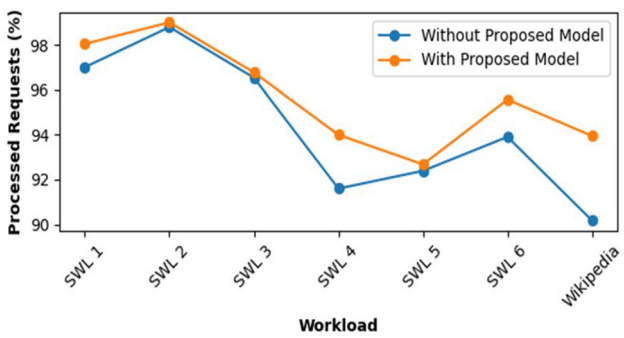



**Fig. 7** Graphical Based Comparison of Processed Requests of Customer Service

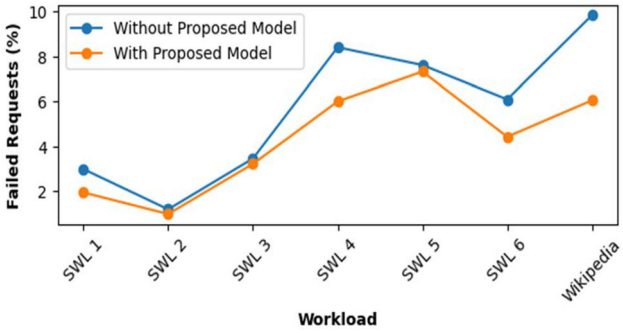



**Fig. 8** Graphical Based Comparison of Failed Requests of Customer Service

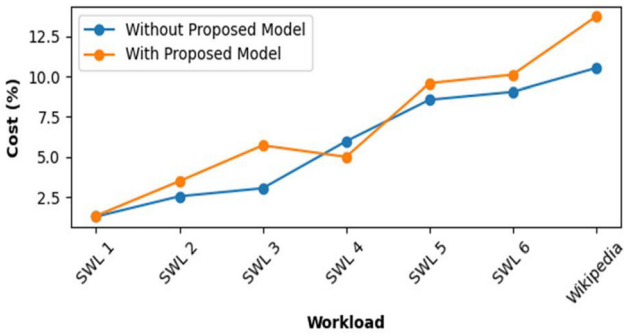



**Fig. 9** Graphical Based Comparison of Cost of Customer Service

successfully without using the proposed model. Only in the case of SWL 6, the proposed framework is leading to a reduced number of successfully processed requests.

The graphical comparison between the failed user requests of order service with and without using the proposed framework is represented in Fig. 14. It is clearly visible from Fig. 14 that the number of failed user requests for order service is reduced using the proposed framework. Only in the case of SWL 6 the proposed framework is resulting in an increased number of failed user requests.

The graphical based comparison between the costs of order service with and without proposed work is shown in Fig. 15. In the case of SWL 1 and SWL 4, the cost is almost the same in both cases and in the case of SWL4, the cost is reduced using the proposed framework. In the rest of the cases, proposed framework is increasing the cost.

The performance of the proposed framework is measured based on three metrics, detailed in Table 10. The first one is Processed requests, which shows the number of requests served successfully within the specified time duration. The second parameter is the number of failed requests which contains all the user requests that were not served successfully within the time duration, no matter the reason for failure. The last attribute is Reliability, representing the reliability of microservice applications with and without the proposed framework. The reliability of both the

**Table 8** Simulation Results of Catalog Service

| Workload | Processed Requests (%) | | Failed Requests (%) | | Cost (%) | |
|---|---|---|---|---|---|---|
| | Without Model | With Model | Without Model | With Model | Without Model | With Model |
| SWL 1 | 97.03 | 98.35 | 2.97 | 1.65 | 1.44 | 1.89 |
| SWL 2 | 98.76 | 98.99 | 1.24 | 1.01 | 2.6 | 3.73 |
| SWL 3 | 97.52 | 98.01 | 2.48 | 1.99 | 3 | 3.19 |
| SWL 4 | 92.59 | 92.59 | 7.41 | 7.41 | 5.96 | 5.74 |
| SWL 5 | 93.39 | 96.89 | 6.61 | 3.11 | 8.35 | 9.68 |
| SWL 6 | 91.68 | 93.19 | 8.32 | 6.81 | 9.87 | 10.23 |
| Wikipedia | 92.31 | 93.96 | 7.69 | 6.04 | 10.66 | 11.46 |

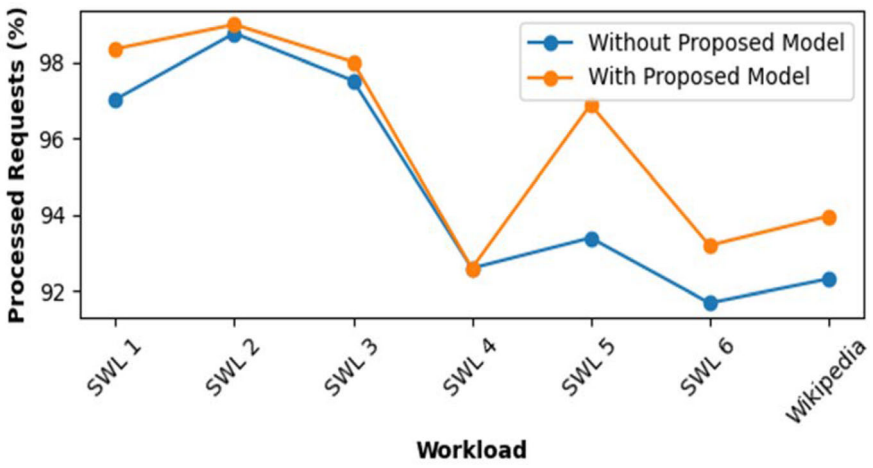



**Fig. 10** Graphical Based Comparison of Processed Requests of Catalog Service

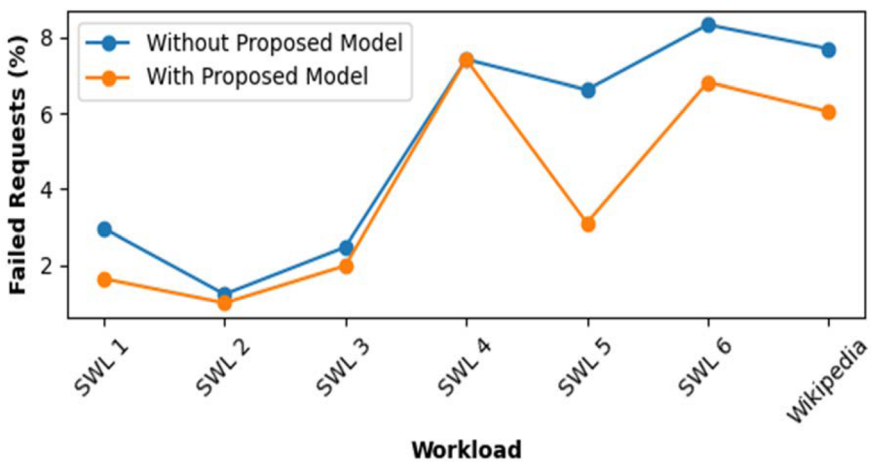



**Fig. 11** Graphical Based Comparison of Failed Requests of Catalog Service

cases is calculated using the formula mentioned in Algorithm 1. The results show that the proposed framework is making the microservice application more reliable in the presence of bursty workloads.

Now the comparison in the performance of case study applications with and without proposed framework is made. A number of successfully processed requests and failed requests contributes to determining the reliability of an application. That is why the next three figures show the comparison between the proposed framework approach and the case study application not using the proposed framework in terms of processed requests, failed requests, and reliability of the complete application.

As shown in Fig. 16, the proposed framework is always leading to an improved number of successfully executed user requests. Only in the case of SWL 5 the number of processed requests in both the cases is almost the same but in all other workload cases the proposed framework results in better performance.

The graphical representation of failed requests of the Microservice Consul application is shown in Fig. 17. The proposed framework resulted in a reduced number of failed requests in all workload cases except in the SWL 5 case. When a synthetic work SWL 5 is applied to the application the number of failed requests in the presence and absence of the proposed framework is almost same.

In Fig. 18, a graphical based comparison of the reliability of complete case study application with and without proposed framework is done. It is clearly visible from this figure that the proposed framework is improving the reliability of the application for different workloads. In case of SWL 5, the reliability of case study application is almost same for both the cases.

After analysing the performance of individual microservices and complete application, it can be concluded that the proposed framework is improving the number of successfully processed user requests and reducing the number of failed requests. This reduction of failed requests in user requests results in improved reliability of the complete application. So we can say that the proposed framework is making the microservice architecture based application more reliable for bursty workloads. However, the cost of the resources is increasing with the proposed approach as resource utilization is high with the proposed approach.

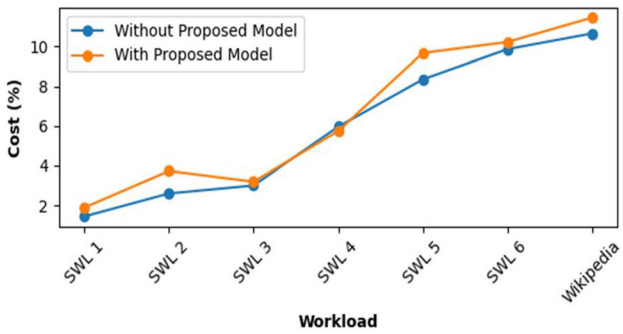



**Fig. 12** Graphical Based Comparison of Cost of Catalog Service

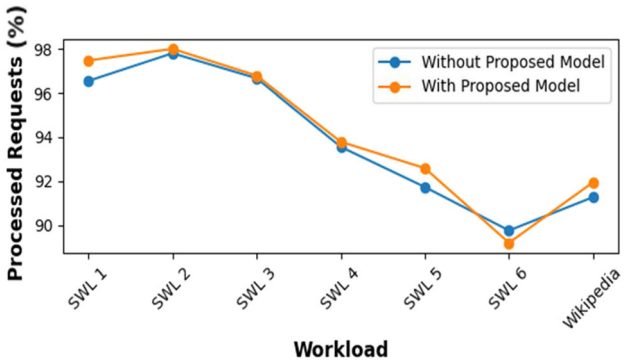



**Fig. 13** Graphical Based Comparison of Processed Requests of Order Service

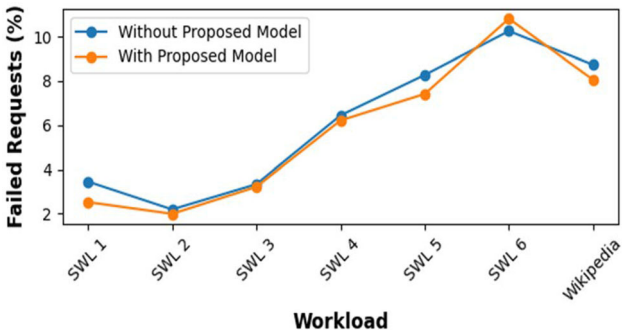



**Fig. 14** Graphical Based Comparison of Failed Requests of Order Service

**Table 9** Simulation Results of Order Service

| Workload | Processed Requests (%) | | Failed Requests (%) | | Cost (%) | |
|---|---|---|---|---|---|---|
| | Without Using Proposed Model | With Proposed Model | Without Using Proposed Model | With Proposed Model | Without Using Proposed Model | With Proposed Model |
| SWL 1 | 96.55 | 97.48 | 3.45 | 2.52 | 1.33 | 1.38 |
| SWL 2 | 97.81 | 98.01 | 2.19 | 1.99 | 2.59 | 3.55 |
| SWL 3 | 96.67 | 96.79 | 3.33 | 3.21 | 3.15 | 5.77 |
| SWL 4 | 93.56 | 93.79 | 6.44 | 6.21 | 5.99 | 5.03 |
| SWL 5 | 91.73 | 92.59 | 8.27 | 7.41 | 8.59 | 9.15 |
| SWL 6 | 89.75 | 89.19 | 10.25 | 10.81 | 9.09 | 9.27 |
| Wikipedia | 91.27 | 91.95 | 8.73 | 8.05 | 10.59 | 11.55 |

## 6 Threats to validity

Several potential threats to validity have been identified in the proposed work. Firstly, the empirical evaluation of the proposed framework is carried out on a limited-scale case study application comprising only three microservices. Hence, it is essential to conduct a more thorough empirical analysis of applications that feature a greater number of microservices to determine the scalability of the proposed framework. As the number of microservices grows, the placement strategy that considers communication overhead may become increasingly time-consuming, necessitating an evaluation of the algorithm's efficacy under such conditions. Also, reliability in microservice applications can be influenced by a myriad of factors, such as communication interruptions due to hardware issues, failure of external dependencies, and deployment strategies, etc. However, this work focused only on effective workload handling and robust microservices communication mechanisms. Furthermore, while the results demonstrate that the proposed approach effectively enhanced the reliability of the microservices application by reducing the number of failed user requests, it also resulted in increased costs compared to the baseline application.

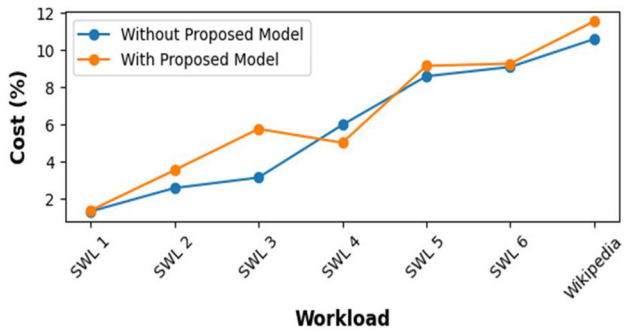



**Fig. 15** Graphical Based Comparison of Cost of Order Service

## 7 Conclusion

Reliability is a crucial Quality of Service (QoS) parameter for any software application as it significantly influences its availability to users in bursty workloads. Conversely, microservices architecture is currently trending and has emerged as a highly favoured approach for designing large-scale enterprise applications. While MSA offers various advantages over other architectural styles, their

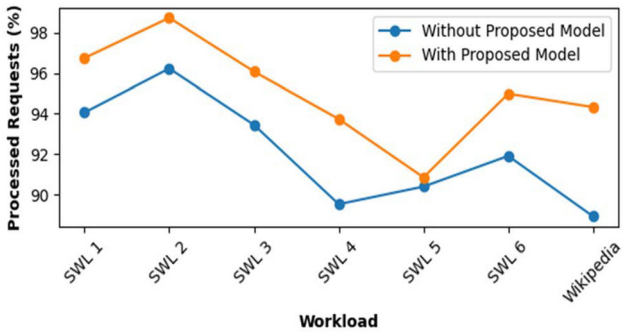



**Fig. 16** Graphical Based Comparison of Processed Requests of Microservice Consul Application

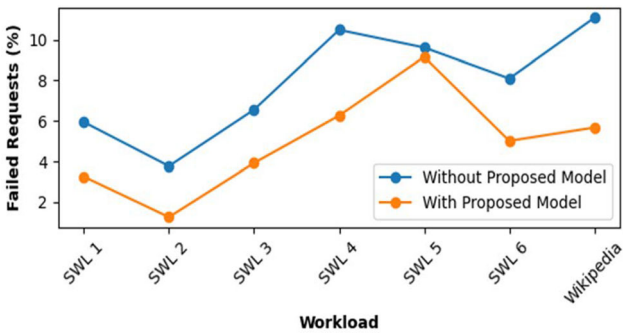



**Fig. 17** Graphical Based Comparison of Failed Requests of Microservice Consul Application

**Table 10** Simulation Results of Microservice Consul Application

| Workload | Processed Requests (%) | | Failed Requests (%) | | Cost (%) | |
|---|---|---|---|---|---|---|
| | Without Using Proposed Model | With Proposed Model | Without Using Proposed Model | With Proposed Model | Without Using Proposed Model | With Proposed Model |
| SWL 1 | 94.06 | 96.75 | 5.94 | 3.25 | 0.94 | 0.96 |
| SWL 2 | 96.23 | 98.74 | 3.77 | 1.26 | 0.96 | 0.98 |
| SWL 3 | 93.45 | 96.08 | 6.55 | 3.92 | 0.93 | 0.96 |
| SWL 4 | 89.52 | 93.74 | 10.48 | 6.26 | 0.89 | 0.93 |
| SWL 5 | 90.39 | 90.85 | 9.61 | 9.15 | 0.90 | 0.90 |
| SWL 6 | 91.91 | 94.99 | 8.09 | 5.01 | 0.91 | 0.94 |
| Wikipedia | 88.92 | 94.33 | 11.08 | 5.67 | 0.88 | 0.94 |

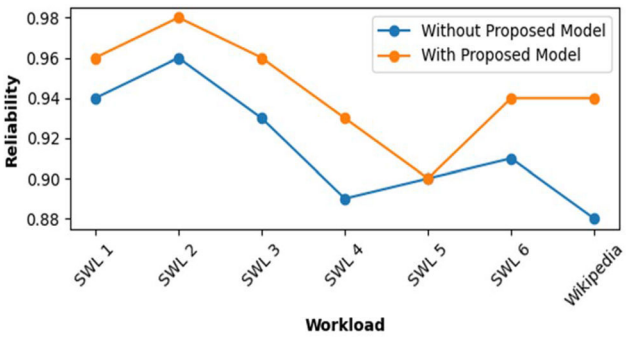



**Fig. 18** Graphical Based Comparison of Reliability of Microservice Consul Application

decentralised and distributed nature can impact application reliability during bursty workloads. This paper introduces a framework aimed at enhancing the reliability of microservices-based applications, with comparative results demonstrating satisfactory outcomes. Because of hardware constraints, a small size case study application is used but in the future, the proposed framework can be tested for a large real time microservices application. The outcomes demonstrate that the proposed framework has increased the cost of the microservices application while improving reliability. So in the future, an approach for the efficient utilization of resources can be integrated with the proposed framework and other techniques like Normalized Entropy (NE) and Fast Fourier Transformation (FFT) can also be explored for burst detection.

**Author contributions** Neha Kaushik has performed the literature review and identified the problem statement. The idea was discussed and validated by Harish Kumar and Vinay Raj. The experimental work, analysis and writing the draft of the manuscript was done by Neha Kaushik. The results were verified by Harish Kumar and Vinay Raj has reviewed the final draft of the paper.

**Funding** Not Applicable.

**Data availability** No datasets were generated or analysed during the current study.

## Declarations

**Conflict of interest** The authors declare no competing interests.

## References

1. Raj, V., Sadam, R.: Patterns for migration of SOA based applications to microservices architecture. J. Web Eng. **20**(5), 1229–1246 (2021)
2. Tapia, F., Mora, M.A., Fuertes, W., Aules, H., Flores, E., Toulkeridis, T.: From monolithic systems to microservices: a comparative study of performance. Appl. Sci. **10**(17), 5797 (2020)
3. Kaushik, N., Kumar, H., Raj, V.: Empirical evaluation of microservices architecture. In International Conference on Communication and Intelligent Systems (pp. 241-253). Springer, Singapore (2022)
4. Kaushik, N., Kumar, H., Raj, V.: Micro frontend based performance improvement and prediction for microservices using machine learning. J. Grid Comput. **22**(2), 1–26 (2024)
5. Lercher, A., Glock, J., Macho, C., Pinzger, M.: Microservice API evolution in practice: a study on strategies and challenges. J. Syst. Software 112110 (2024)
6. Wang, Y., Kadiyala, H., Rubin, J.: Promises and challenges of microservices: an exploratory study. Empir. Softw. Eng. **26**(4), 63 (2021)
7. Di Francesco, P., Lago, P., Malavolta, I.: Architecting with microservices: a systematic mapping study. J. Syst. Softw. **150**, 77–97 (2019)
8. Li, S., Zhang, H., Jia, Z., Zhong, C., Zhang, C., Shan, Z., Babar, M.A.: Understanding and addressing quality attributes of microservices architecture: a systematic literature review. Inform. Software Technol. **131**, 106449 (2021)
9. Jagadeesan, L. J., Mendiratta, V. B.: When failure is (not) an option: reliability models for microservices architectures. In 2020 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), pp. 19–24. IEEE (2020)
10. Shafi, N., Abdullah, M., Iqbal, W., Erradi, A., Bukhari, F.: 2024. Cdascaler: a cost-effective dynamic autoscaling approach for containerized microservices. Cluster Comput. pp.1–21
11. Wang, H., Wang, L., Yu, Q., Zheng, Z., Bouguettaya, A., Lyu, M.R.: Online reliability prediction via motifs-based dynamic Bayesian networks for service-oriented systems. IEEE Trans. Software Eng. **43**(6), 556–579 (2016)
12. Abdullah, M., Iqbal, W., Berral, J.L., Polo, J., Carrera, D.: Burst-aware predictive autoscaling for containerized microservices. IEEE Trans. Serv. Comput. **15**(3), 1448–1460 (2020)
13. Li, Z., Yu, H., Fan, G., Zhang, J.: Cost-efficient fault-tolerant workflow scheduling for deadline-constrained microservice-based applications in clouds. IEEE Transactions on Network and Service Management (2023)
14. Xie, S., Wang, J., Li, B., Zhang, Z., Li, D., Hung, P.C.: PBScaler: a bottleneck-aware autoscaling framework for microservice-based applications. IEEE Transactions on Services Computing (2024)
15. Ding, Z., Xu, T., Ye, T., Zhou, Y.: Online prediction and improvement of reliability for service oriented systems. IEEE Trans. Reliab. **65**(3), 1133–1148 (2015)
16. El Malki, A., Zdun, U., Pautasso, C.: Impact of API rate limit on reliability of microservices-based architectures. In 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), pp. 19–28. IEEE (2022)
17. Pietrantuono, R., Russo, S., Guerriero, A.: Testing microservice architectures for operational reliability. Software Test. Verific. Reliab. **30**(2), e1725 (2020)
18. Liu, Z., Yu, H., Fan, G., Chen, L.: Reliability modelling and optimization for microservice based cloud application using multiagent system. IET Commun. **16**(10), 1182–1199 (2022)
19. Song, Z., Tilevich, E.: Equivalence-enhanced microservice workflow orchestration to efficiently increase reliability. In 2019 IEEE International Conference on Web Services (ICWS), pp. 426–433. IEEE (2019)
20. Camilli, M., Guerriero, A., Janes, A., Russo, B., Russo, S.: Microservices integrated performance and reliability testing. In Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test, pp. 29–39 (2022)
21. Wang, H., Yang, Z., Yu, Q., Hong, T., Lin, X.: Online reliability time series prediction via convolutional neural network and long

short term memory for service-oriented systems. Knowl.-Based Syst. **159**, 132–147 (2018)
22. Yan, M., Liang, X., Lu, Z., Wu, J., Zhang, W.: HANSEL: adaptive horizontal scaling of microservices using Bi-LSTM. Appl. Soft Comput. **105**, 107216 (2021)
23. Feng, B., Ding, Z., Jiang, C.: FAST: a forecasting model with adaptive sliding window and time locality integration for dynamic cloud workloads. IEEE Trans. Serv. Comput. **16**(2), 1184–1197 (2022)
24. Xu, M., Song, C., Ilager, S., Gill, S.S., Zhao, J., Ye, K., Xu, C.: CoScal: multifaceted scaling of microservices with reinforcement learning. IEEE Trans. Netw. Serv. Manage. **19**(4), 3995–4009 (2022)
25. Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., He, J., Xu, C.: An in-depth study of microservice call graph and runtime performance. IEEE Trans. Parallel Distrib. Syst. **33**(12), 3901–3914 (2022)
26. Joseph, C.T., Chandrasekaran, K.: IntMA: dynamic interaction-aware resource allocation for containerized microservices in cloud environments. J. Syst. Architect. **111**, 101785 (2020)
27. Iqbal, W., Dailey, M.N., Carrera, D., Janecek, P.: Adaptive resource provisioning for read intensive multi-tier applications in the cloud. Futur. Gener. Comput. Syst. **27**(6), 871–879 (2011)
28. Tahir, F., Abdullah, M., Bukhari, F., Almustafa, K.M., Iqbal, W.: Online workload burst detection for efficient predictive autoscaling of applications. IEEE Access **8**, 73730–73745 (2020)
29. Stefan, S., Niculescu, V.: Microservice-oriented workload prediction using deep learning. e-Informatica Software Eng. J. **16**(1) (2022)
30. Raj, V., Sadam, R.: Evaluation of SOA-based web services and microservices architecture using complexity metrics. SN Comput. Sci. **2**, 1–10 (2021)

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



**Neha Kaushik** is a Ph.D. scholar in Department of Computer Engineering, J. C. Bose University of Science and Technology, Faridabad, India. She has completed her B.Tech[CSE] from MDU, Rohtak, and M.Tech [CE] from JCBUST, Faridabad. Her research interest includes Service Oriented Architectures, Microservices, Machine Learning.



**Harish Kumar** is currently working as Associate Professor in Department of Computer Engineering, J. C. Bose University of Science and Technology, Faridabad, India. His research interest includes Microservices, Software testing, Machine Learning.



**Vinay Raj** is currently working as Assistant Professor in the Department of Computer Applications, NIT Tiruchirappalli. He completed his full-time Ph.D. from NIT Warangal and M.Tech from BITS Pilani. His research interests include Software architectures, Microservices, Serverless Computing, Cloud Computing and Machine Learning. He has worked as Senior software Engineer in Capgemini and Software Developer in TCS, Hyderabad.